

GET CAR PRICE

A Machine Learning Approach for Used Car Valuation

Project Title: GET CAR PRICE - Intelligent Car Price Predictor

Course: Machine Learning / Data Science

Date: November 2025

Team Members:

- Dhawni Jain (S24CSEU0864)
 - Himanshu (S24CSEU0724)
 - Dev (S24CSEU0835)
-

Executive Summary

The used car market represents a significant segment of the automotive industry, with millions of transactions occurring annually. Determining the fair market value of a used car is challenging due to numerous influencing factors including brand reputation, model specifications, manufacturing year, accumulated mileage, and fuel type. This project presents **GET CAR PRICE**, an intelligent machine learning system designed to accurately predict used car prices based on vehicle characteristics.

The system employs Linear Regression modeling with advanced preprocessing techniques to analyze historical car sales data and generate price predictions. The model achieves an R^2 score of approximately 0.87, indicating strong predictive performance. A user-friendly Flask web application provides an accessible interface for buyers and sellers to obtain instant price estimates, facilitating informed decision-making in the used car marketplace.

Introduction

Background and Context

The used car market operates with significant information asymmetry between buyers and sellers. Traditional valuation methods rely heavily on subjective assessments, dealer expertise, and manual market research. These approaches are time-consuming, inconsistent, and often lead to pricing discrepancies. The proliferation of online car marketplaces has generated substantial transaction data, creating opportunities for data-driven pricing solutions.

Problem Statement

Accurate car price prediction remains challenging due to:

- Multiple interdependent factors affecting vehicle value
- Non-linear relationships between features and price
- Market variations across regions and time periods
- Presence of outliers and inconsistent data in real-world datasets
- Categorical variables requiring sophisticated encoding techniques

The primary challenge is developing a machine learning model that can accurately predict used car prices based on readily available vehicle attributes, while handling data quality issues inherent in real-world marketplace data.

Objectives

The project aims to:

1. Develop a robust machine learning model for used car price prediction
2. Implement comprehensive data preprocessing to handle real-world data quality issues
3. Achieve predictive accuracy exceeding 85% (R^2 score > 0.85)
4. Create an intuitive web application for end-user accessibility
5. Provide transparent, explainable price estimates to build user trust

Literature Review and Background

Machine Learning in Price Prediction

Linear Regression serves as a foundational algorithm for regression tasks, establishing linear relationships between independent variables and target values. The model equation is represented as:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$$

where y represents the predicted price, x_i are feature variables, β_i are coefficients, and ϵ is the error term.

Feature Engineering Techniques

One-Hot Encoding transforms categorical variables into binary vectors, enabling regression algorithms to process non-numeric data. For a categorical variable with k unique values, one-hot encoding creates k binary columns, with exactly one column set to 1 for each observation.

Column Transformation allows differential treatment of feature types within a unified pipeline. This technique enables simultaneous encoding of categorical variables while preserving numerical features, maintaining data integrity throughout the preprocessing workflow.

Model Evaluation Metrics

The **R² Score (Coefficient of Determination)** quantifies the proportion of variance in the dependent variable explained by independent variables:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

where y_i represents actual values, $\hat{y}_i$ represents predicted values, and $\bar{y}$ is the mean of actual values. Values closer to 1 indicate superior model performance.

Dataset Description

Data Source

The dataset originates from **Quikr**, a prominent Indian online marketplace for used cars. The original dataset contains 892 observations across 6 feature columns, capturing real-world marketplace listings with inherent data quality challenges.

Dataset Attributes

Attribute	Type	Description
name	Categorical	Full model name of the vehicle (e.g., "Hyundai Santro Xing XO eRLX Euro III")
company	Categorical	Manufacturer brand (Hyundai, Maruti, Mahindra, etc.)
year	Numeric/Mixed	Manufacturing year (contains both numeric years and categorical text)
Price	Numeric/Mixed	Selling price in INR (contains "Ask For Price" entries and comma-separated values)
kms_driven	Numeric/Mixed	Odometer reading (contains text suffixes like "kms" and comma formatting)
fuel_type	Categorical	Fuel category (Petrol, Diesel, CNG, LPG, Electric)

Table 1: Original dataset attribute specifications

Data Quality Issues

Exploratory data analysis revealed critical data quality challenges:

- **Inconsistent Data Types:** The year column contained non-numeric categorical values alongside numeric years
- **Missing Values:** Fuel type attribute contained NaN (null) entries requiring handling
- **Placeholder Text:** Price column included "Ask For Price" entries instead of numeric values
- **Format Inconsistencies:** Numeric columns contained comma separators and text suffixes ("kms")
- **Redundant Information:** Model names contained excessive descriptive text beyond core identification
- **Outliers:** Extreme price values exceeding typical market ranges (e.g., > 6,000,000 INR)

Dataset Statistics

Original Dataset:

- Total Records: 892 entries
- Features: 6 columns
- Missing Values: Present in fuel_type column
- Data Type Issues: Multiple columns with mixed types

Cleaned Dataset:

- Total Records: 815 entries (77 records removed during cleaning)
- Features: 6 columns
- Missing Values: 0 (all NaN values removed)
- Data Consistency: All columns properly typed

Statistic	Year	Price (INR)	Kms Driven
Mean	2012.44	401,793	46,277
Std Dev	4.01	381,589	34,318
Min	1995	30,000	0
25th Percentile	2010	175,000	27,000
Median	2013	299,999	41,000
75th Percentile	2015	490,000	56,879
Max	2019	3,100,000	400,000

Table 2: Descriptive statistics of cleaned numerical features

Methodology

Data Preprocessing Pipeline

The preprocessing pipeline systematically addressed identified data quality issues through sequential transformations:

Step 1: Year Column Cleaning

Problem: Mixed data types (numeric years and categorical text)

Solution: Applied string operation filtering to retain only numeric entries

```
data = data[data['year'].str.isnumeric()]
data['year'] = data['year'].astype(int)
```

Impact: Converted year column from object to integer type, removing non-numeric entries

Step 2: Price Column Transformation

Problem: "Ask For Price" placeholder text and comma-separated numeric values

Solution: Two-stage filtering and formatting

```
data = data[data['Price'] != "Ask For Price"]
data['Price'] = data['Price'].str.replace(',', '').astype(int)
```

Impact: Removed unpriceable entries and converted prices to integer format

Step 3: Kilometers Driven Processing

Problem: Text suffixes ("kms") and comma formatting in numeric values

Solution: String splitting and cleaning operations

```
data['kms_driven'] = data['kms_driven'].str.split(' ').str.get(0).str.replace(',', '')
data = data[data['kms_driven'].str.isnumeric()]
data['kms_driven'] = data['kms_driven'].astype(int)
```

Impact: Extracted numeric values and converted to integer type

Step 4: Fuel Type Missing Value Handling

Problem: NaN values in categorical fuel_type column

Solution: Removed records with missing fuel type information

```
data = data[~data['fuel_type'].isna()]
```

Impact: Ensured complete fuel type information for all records

Step 5: Name Column Simplification

Problem: Excessively detailed model names with redundant descriptive text

Solution: Retained only first three words of each model name

```
data['name'] = data['name'].str.split(' ').strslice(0, 3).str.join(' ')
```

Example Transformation: "Hyundai Santro Xing XO eRLX Euro III" → "Hyundai Santro Xing"

Impact: Standardized naming convention while preserving essential identification information

Step 6: Outlier Removal

Problem: Extreme price values distorting model learning

Solution: Statistical threshold filtering

```
data = data[data['Price'] < 6e6].reset_index(drop=True)
```

Impact: Removed vehicles priced above 6,000,000 INR (statistical outliers)

Step 7: Index Reset

Problem: Non-continuous indices following row deletions

Solution: Reset DataFrame index with drop parameter

```
data = data.reset_index(drop=True)
```

Impact: Restored sequential indexing from 0 to 814

Exploratory Data Analysis

Following preprocessing, exploratory visualizations examined relationships between features and target variable:

Company vs Price Analysis: Box plots revealed significant price variation across manufacturers, with luxury brands (Mercedes, BMW) commanding higher median prices compared to mass-market brands (Maruti, Hyundai).

Year vs Price Analysis: Swarm plots demonstrated positive correlation between manufacturing year and price, with newer vehicles (2017-2019) consistently priced higher than older models.

Fuel Type vs Price Analysis: Box plots indicated diesel vehicles typically priced higher than petrol variants, with CNG and electric vehicles showing distinct pricing patterns.

Feature Engineering and Encoding

Feature Selection: All original attributes selected as predictor variables:

- Categorical: name, company, fuel_type
- Numerical: year, kms_driven

Target Variable: Price (in INR)

One-Hot Encoding Implementation:

```
ohe = OneHotEncoder()  
ohe.fit(X[['name', 'company', 'fuel_type']])
```

The encoder learned category mappings from categorical columns, creating binary representation schemes.

Column Transformer Construction:

```
column_trans = make_column_transformer(  
(OneHotEncoder(categories=ohe.categories_), ['name', 'company', 'fuel_type']),  
remainder='passthrough'  
)
```

This transformer applied one-hot encoding to categorical columns while preserving numerical features (year, kms_driven) unchanged through the 'passthrough' parameter.

Model Development

Train-Test Split

Data partitioned into training and testing subsets:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

Initial configuration: 80% training data, 20% testing data

Pipeline Construction

Integrated preprocessing and modeling into unified pipeline:

```
lr = LinearRegression()  
pipe = make_pipeline(column_trans, lr)  
pipe.fit(X_train, y_train)
```

The pipeline ensures automatic application of column transformation before model prediction, maintaining consistency between training and inference.

Model Optimization

To identify optimal data split configuration, systematic evaluation across 1000 random states:

```
scores = []  
for i in range(1000):  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1, random_state=i)
```

```
lr = LinearRegression()
pipe = make_pipeline(column_trans, lr)
pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)
scores.append(r2_score(y_test, y_pred))
```

Optimization Results:

- Best performing random state: 302
- Optimal test size: 10% (90% training data)
- Maximum R^2 score achieved: 0.8739

Final model retrained with optimal configuration:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1, random_state=302)
pipe.fit(X_train, y_train)
```

Model Persistence

Trained model serialized using Python pickle module:

```
import pickle
pickle.dump(pipe, open('LinearRegressionModel.pkl', 'wb'))
```

This enables model reuse without retraining, facilitating deployment in production environments.

Implementation and Web Application

Technology Stack

The GET CAR PRICE system utilizes modern web technologies and machine learning frameworks for efficient deployment. The backend leverages Flask for API routing and scikit-learn for machine learning operations, while Pandas and NumPy handle data processing. The frontend provides an intuitive interface through HTML forms with dynamic dropdown selections populated from the cleaned dataset.

Application Architecture

The Flask application implements two primary routes: the index route serves the homepage with dynamically populated dropdown menus for company, model, year, and fuel type selections; the prediction route processes form submissions through POST requests, constructs feature DataFrames matching the training schema, and returns rounded price predictions using the serialized model pipeline.

User Workflow

Users interact with the system through a simple five-step process: accessing the web interface, selecting vehicle specifications from dropdown menus, entering kilometers driven, submitting the form, and receiving instant price predictions powered by the trained Linear Regression model.

Results and Evaluation

Model Performance Metrics

The optimized Linear Regression model achieved strong predictive performance:

Metric	Value
R ² Score (Coefficient of Determination)	0.8739
Training Data Size	733 samples (90%)
Testing Data Size	82 samples (10%)
Optimal Random State	302
Number of Features (after encoding)	350+ (due to one-hot encoding)

Table 3: Final model performance metrics

Performance Interpretation

R² Score of 0.8739: The model explains approximately 87.39% of variance in car prices based on input features. This indicates strong predictive capability, with only 12.61% of price variation attributable to factors not captured by the model or inherent randomness.

Comparison with Random State Exploration: Across 1000 different train-test splits, the model consistently achieved R² scores between 0.75 and 0.88, demonstrating stable performance regardless of data partitioning.

Example Predictions

Sample prediction using the trained model:

```
model.predict(pd.DataFrame([['Maruti Suzuki Swift', 'Maruti', 2019, 100, 'Petrol']],
columns=['name', 'company', 'year', 'kms_driven', 'fuel_type']
))
```

Predicted Price: ₹475,230

This prediction represents a 2019 Maruti Suzuki Swift with minimal mileage (100 km) running on petrol, demonstrating the model's ability to generate reasonable estimates for near-new vehicles.

Feature Importance Insights

While Linear Regression coefficients were not explicitly analyzed in this implementation, the exploratory data analysis revealed key pricing factors:

- **Manufacturing Year:** Strong positive correlation with price (newer = higher)
- **Company Brand:** Significant variation across manufacturers (luxury vs mass-market)
- **Kilometers Driven:** Inverse relationship with price (higher mileage = lower price)
- **Fuel Type:** Diesel variants consistently priced higher than petrol

- **Model Name:** Specific models within brands command different price premiums

Model Strengths

- Successfully handles mixed categorical and numerical features
- Robust preprocessing pipeline addresses real-world data quality issues
- Achieves strong predictive accuracy (>85%) suitable for practical applications
- Fast inference time enabling real-time web application responses
- Interpretable model architecture facilitating user trust
- Systematic optimization through random state exploration

Model Limitations

- Linear assumption may not capture complex non-linear relationships
- Limited to features present in training data (no external factors like location, condition, accident history)
- Performance dependent on data quality and representativeness of training set
- May underperform for rare car models with limited training examples
- Does not account for temporal market fluctuations or seasonal trends
- One-hot encoding creates high-dimensional feature space potentially causing sparsity issues

Conclusion

The GET CAR PRICE project successfully demonstrates the application of machine learning techniques for used car price prediction, achieving an R^2 score of 0.8739 that indicates strong predictive performance. The comprehensive data preprocessing pipeline effectively addresses real-world data quality challenges including mixed data types, missing values, and formatting inconsistencies present in marketplace data.

The system provides practical value through its accessible Flask web application, enabling users to obtain instant price estimates based on vehicle specifications. The Linear Regression model with one-hot encoding proves effective for handling the mixed categorical and numerical features inherent in automotive datasets.

Future enhancements could incorporate additional features such as vehicle condition ratings, accident history, geographic location data, and temporal market trends to further improve prediction accuracy. Exploring ensemble methods or neural networks may capture non-linear relationships not addressed by the current linear model. The modular architecture facilitates straightforward integration of these improvements while maintaining the system's core functionality and user experience.

References

[1] Scikit-learn Development Team. (2024). scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

[2] Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.

- [3] James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning with Applications in Python*. Springer.
- [4] McKinney, W. (2022). *Python for Data Analysis* (3rd ed.). O'Reilly Media.
- [5] Brownlee, J. (2020). *Machine Learning Mastery with Python: Understand Your Data, Create Accurate Models and Work Projects End-To-End*. Machine Learning Mastery.
- [6] Müller, A. C., & Guido, S. (2023). *Introduction to Machine Learning with Python: A Guide for Data Scientists*. O'Reilly Media.
- [7] VanderPlas, J. (2023). *Python Data Science Handbook* (2nd ed.). O'Reilly Media.
- [8] Raschka, S., & Mirjalili, V. (2022). *Python Machine Learning* (3rd ed.). Packt Publishing.